

Why Build a Socratic AI Tutor

The case, the evidence, and the architecture

James Martin II, PhD

It is not whether students use AI. It is how.

Students already reach for AI to study. The interesting question is no longer adoption, it is what the tool does to their thinking. That turns out to be a design decision, and the design decides whether AI strengthens learning or quietly erodes it.

The trap

Cognitive offloading cuts both ways. When students offload the busywork, formatting, reorganizing, drafting a table, they free working memory for the real thinking, and the learning still happens in their heads. When they offload the thinking itself, the tool does the reasoning they needed to do, and the learning never lands. Same tool: the difference is what they hand it.

The risk is not hypothetical. In a roughly 1,000-student randomized trial, students using AI did better during practice, then worse than the no-AI group once the AI was removed. Guardrails that gave hints instead of answers reduced the effect. Two quieter costs compound it: leaning on AI weakens a student's ability to notice what they do and do not understand, and fluency is mistaken for knowledge, so a student who sounds right assumes they are.

Why this matters

Bastani et al., 2024 (PNAS): the performance paradox, where AI help during practice reverses once it is withdrawn. Lodge and Loble, 2026: the same pattern. Fan et al., 2025: "metacognitive laziness," reduced self-monitoring when leaning on AI. The moment the AI is gone is exactly boards and the patient.

What actually works

The learning-science evidence points in one direction. Of the common study techniques, retrieval practice and distributed practice rank highest, and they share one feature: the student has to produce, not receive. Retrieving information from memory does not just measure learning, it strengthens it. In medical education specifically, repeated testing beat repeated study at six months, and self-directed retrieval practice predicted higher Step 1 scores. A good tutor is built around that single mechanism.

The evidence base

Dunlosky et al., 2013: practice testing and spacing are the two highest-utility strategies. Roediger and Karpicke, 2006: the testing effect. Larsen et al., 2009: test-enhanced learning held at six months in a real teaching setting. Deng et al., 2015: self-directed retrieval predicted Step 1 performance.

The bridge

A Socratic tutor operationalizes the "how." It withholds the answer, so the student produces first. It forces retrieval, so every turn is a testing event. It builds metacognition, because it surfaces what the student cannot yet do rather than papering over it. This is not a stylistic preference: a Socratic chatbot outperformed a standard chatbot on reflection and critical thinking (Favero et al., 2024). The same evidence that indicts answer-machine use endorses Socratic design.

The proof

We piloted this with Professor Dobbi, a Socratic tutor, in Fall 2025 (MBS Biochemistry, N = 58, voluntary use, no training provided). The average did not move in a statistically significant way ($p = 0.32$), and that is the wrong place to look. The distribution moved.

down 67%

D/F rate (17.4% to 5.7%)

2x

A grades roughly doubled

SD 13.0 to 10.0

spread compressed

Adoption was U-shaped: at-risk students 87%, strong students 71%, the middle 38%. The effect split by habit. Strong students gained about +4.8, while students who used the tutor as an answer machine lost about 4.9, and heavy use did not save them. The tool did not decide the outcome. The habit did. That is precisely why the architecture matters: if how you use it decides the result, then a design that enforces the right how is the whole game.

The how: architecture, not vibes

Four things separate a tutor from a chatbot. None of them are about the subject matter, which is why they transfer to any course.

1. **Session state.** A running ledger of what is unresolved, so the tutor teaches across turns instead of answering one-off questions.
2. **The evaluation gate.** It classifies every answer. It never praises a wrong one, and never hands the answer on a first miss.
3. **Two personas.** Warmth to keep the student engaged, and a sharp challenger so comfort never becomes complacency.
4. **Protected practice.** It builds the skill on adjacent problems without ever spoiling the graded case.

One rule governs all four: never praise a wrong answer, and never hand over the answer on a first miss. A student who receives warmth in place of a correction has been made less safe, not more, because they carry the error into a patient encounter. Psychological safety comes from accurate feedback, not from its absence.

The on-ramp

Nobody has to start from a blank system prompt. Start with TRACI (Task, Role, Audience, Create, Intent), which faculty already use to structure a prompt. Add the four principles, which is the jump from prompt to tutor. Then deploy platform-agnostically: a Gemini Gem, a Claude Project, a custom GPT, or NotebookLM. The method does not depend on the platform.

What to share, what to keep

Share the method. Teaching others to build these tutors is the work that builds people up, and a framework earns citations where a copied prompt earns nothing. Keep your fully built, validated architecture as yours to deploy and to extend. Students do not need a faster way to get answers. They need a partner that makes them think, and that is what a Socratic tutor is for.

Companion to the presentation of the same title. Findings are paraphrased from the cited literature; verify figures against the source manuscripts before external use.